Compra Assignment Reference Document

Chat-Based Layout Agent:

Table of Contents

---

1. Project Objective

What You're Building

You are building a **chat-based layout agent** — a web application where a user can:

- See a design layout (defined by a JSON file containing shapes, images, text, positions, sizes).
- Chat with an AI assistant using natural language (e.g., "Convert this to 9:16", "Make the headline smaller").
- Watch the layout JSON automatically update based on those instructions.
- See a basic wireframe preview of the layout (optional but recommended).

Why It Matters

This project tests your ability to:

- Build a **chat interface** (frontend)
- Integrate with an **LLM** (like Claude or GPT)
- Perform **layout reasoning** (understanding "top", "bigger", "9:16" in design context)
- Handle **JSON transformation** safely and correctly
- Maintain **conversation context** for follow-up instructions

Final Deliverables

- A GitHub repository with working code
- A README with setup instructions
- A short note explaining your approach
- (Optional) A Loom video walkthrough

---

2. Technical Requirements

Recommended Tech Stack

Layer	Tool	Why
Frontend	React + Vite (or Next.js)	Fast setup, chat UI is easy
Styling	Tailwind CSS	Quick clean UI
Backend	Node.js + Express (or Next.js API routes)	Handles LLM calls securely
LLM	Claude API (Anthropic) or OpenAI GPT-4	Natural language understanding
State Management	React useState / Zustand	Track JSON + chat history
Preview	HTML Canvas or absolute-positioned divs	Render wireframe

System Requirements

- Node.js v18 or newer
- npm or yarn
- A code editor (VS Code recommended)
- An API key from Anthropic (Claude) or OpenAI
- Git installed
- A GitHub account

Knowledge Prerequisites

- Basic JavaScript / React
- Understanding of JSON
- Basic familiarity with REST APIs
- Comfort with the command line

---

3. Understanding the Problem

Before coding, spend 30 minutes understanding the JSON structure. This is the most important step.

The Layout JSON Has Three Key Parts

A. The Artboard (Canvas)

- The root container with `width` and `height` (e.g., 1080×1080 for Instagram Post).
- All children are positioned inside this canvas.

B. Nodes (Layers) Each node represents one element (image, text, shape) and contains:

- `x`, `y` — absolute pixel position
- `width`, `height` — absolute pixel size
- `nx`, `ny`, `nw`, `nh` — **normalized** values (0 to 1) relative to the artboard
- `type` — image, text, shape, or artboard
- `data` — content (text string, image URL, etc.)
- `style` — colors, font size, fill, stroke

C. Semantic Roles (Implied) Look at node names and content to identify roles:

- "Product.png" → the main product image
- "Limited time offer" → offer badge / CTA
- "Luxury Comfort, Surprisingly Attainable" → headline
- "20% OFF" → discount badge
- "Background.png" → background

Key Insight: Normalized Coordinates

Notice every node has both `x/y/width/height` AND `nx/ny/nw/nh`. The normalized values (0 to 1) let you **resize the canvas** without breaking the layout. This is your superpower for "Convert to 9:16" requests.

Example: if `nx = 0.3` on a 1080-wide canvas, the new x on a 720-wide canvas is `0.3 × 720 = 216`.

---

4. Step-by-Step Implementation Guide

Step 1: Set Up the Project (30 min)

1. Create a new folder: `mkdir layout-agent && cd layout-agent`
2. Initialize the frontend: `npm create vite@latest client -- --template react`
3. Initialize the backend: `mkdir server && cd server && npm init -y`
4. Install backend dependencies: `npm install express cors dotenv @anthropic-ai/sdk`
5. Install frontend dependencies in `client/`: `npm install axios tailwindcss`

Create a `.env` file in `server/` for your API key:

ANTHROPIC_API_KEY=your_key_herePORT=3001

6.
7. Add `.env` to `.gitignore` immediately. Never commit API keys.

Step 2: Load and Display the Initial JSON (1 hour)

Goal: Show the layout JSON in the UI when the app loads.

1. Save the provided design JSON as `client/src/data/initialLayout.json`.
2. In your main App component, import it and store it in `useState`.
3. Build a simple two-column layout:
   - Left column: Chat window
   - Right column: JSON viewer + wireframe preview
4. Use `<pre>{JSON.stringify(layout, null, 2)}</pre>` to display the JSON.

Step 3: Build the Chat Interface (1.5 hours)

Goal: Let the user type messages and see them displayed.

Components to build:

- `ChatWindow` — scrollable area showing past messages
- `MessageBubble` — single message (user or assistant)
- `ChatInput` — text box + send button

Key state to manage:

- `messages` — array of `{role: "user" | "assistant", content: "..."}`
- `isLoading` — disable input while waiting for the LLM
- `layout` — the current JSON

When the user submits a message:

1. Add their message to the `messages` array.
2. Send the message + current layout to your backend.
3. When the response comes back, update the layout and append the assistant's reply.

Step 4: Build the Backend LLM Endpoint (2 hours)

Goal: A single API endpoint that takes the user's instruction + current JSON and returns updated JSON.

Create a route `POST /api/chat` that:

1. Receives `{message, layout, history}` from the frontend.
2. Constructs a careful **system prompt** for the LLM (see Example Code Section).
3. Calls the Anthropic Claude API with the prompt.
4. Parses the LLM response — it should return both updated JSON and a friendly explanation.
5. Validates the returned JSON structure before sending it back.
6. Returns `{updatedLayout, assistantMessage}` to the frontend.

Critical: Never trust LLM output blindly. Always validate that the returned JSON has the expected shape (rootNodes, nodes, required keys).

Step 5: Design the System Prompt (1.5 hours — most important step)

The quality of your prompt determines the quality of your agent. Your prompt should include:

1. **Role definition** — "You are a layout transformation agent."
2. **JSON schema explanation** — explain `nx/ny/nw/nh` and the node types.
3. **Semantic role hints** — tell the LLM how to identify headline, product, badge, etc., from names/content.
4. **Transformation rules** — for example:
    - "When converting aspect ratios, change the artboard width/height and recompute x/y/width/height for every child using nx/ny/nw/nh."
    - "When moving an element, update both absolute and normalized coordinates."
    - "When resizing text, change `fontSize` in style.visual."
5. **Output format** — strict JSON with a known schema, no commentary outside the JSON.
6. **The current layout JSON** — pass it as context.

Step 6: Implement Core Transformations (2 hours)

Some operations are too risky to leave entirely to the LLM. Build helper functions for the most common ones:

A. `resizeArtboard(layout, newWidth, newHeight)`

- Update the artboard width/height.
- For each child, recompute absolute `x`, `y`, `width`, `height` from normalized values.
- This handles "Convert to 9:16" perfectly.

B. `moveNode(layout, nodeId, position)`

- Position can be "top", "bottom", "center", "left", "right".
- Update both absolute and normalized coordinates.

C. `resizeNode(layout, nodeId, scale)`

- Multiply width, height, and font size by the scale factor.

The LLM can call these as "tools" or you can have it return an action object (e.g., `{action: "resize_artboard", width: 1080, height: 1920}`) that your backend executes.

Step 7: Handle Aspect Ratio Conversion (1 hour)

This is the most common request. Common aspect ratios:

Ratio	Width	Height
1:1 (Instagram Post)	1080	1080
9:16 (Story / Reel)	1080	1920
16:9 (YouTube)	1920	1080
4:5 (Instagram Portrait)	1080	1350

When the canvas changes shape, simple normalized scaling stretches everything. For better results:

- **Width-anchored elements** (full-width backgrounds): scale to fill width.
- **Center-anchored elements** (product, headline): keep them centered, scale proportionally.
- **Corner-anchored elements** (badges, logos): keep them in their corner.

This is where semantic understanding helps the LLM make better choices than pure math.

Step 8: Build the Wireframe Preview (1.5 hours — optional but impressive)

Goal: Show a visual approximation of the layout.

Approach: use absolute-positioned `<div>` elements scaled to fit a preview area.

For each node in the layout:

- Compute its position and size as a percentage of the artboard (use `nx`, `ny`, `nw`, `nh`).
- Render a colored rectangle for shapes, a placeholder for images, and a styled text element for text nodes.
- Use the node's `name` or `content` as a label.

The preview should update automatically whenever the layout JSON updates.

Step 9: Handle Follow-up Instructions (1 hour)

Goal: "Make it smaller" should refer to the last-mentioned element.

How to do this:

1. Always pass the **last N messages** (e.g., 6) to the LLM as conversation history.
2. The LLM uses this context to resolve references like "it", "that", "the headline".
3. Optionally track the "last modified element" in state and pass it as a hint.

Step 10: Polish and Test (1.5 hours)

Test these instructions and make sure each one works:

- "Convert this design to 9:16"
- "Keep the product large" (follow-up)
- "Move the headline to the top"
- "Move the offer badge higher"
- "Make the headline smaller"
- "Change the headline color to red"
- "Make the discount badge bigger"
- "Center the product"

For each, verify:

- The JSON updates correctly
- The wireframe reflects the change
- The chat shows a friendly assistant message
- Errors are handled gracefully (LLM returns invalid JSON, network fails, etc.)

---

5. Project Structure

```
layout-agent/
├── client/                    # Frontend
│   ├── src/
│   │   ├── components/
│   │   │   ├── ChatWindow.jsx       # Scrollable chat messages
│   │   │   ├── MessageBubble.jsx    # Single message
│   │   │   ├── ChatInput.jsx        # Text input + send
│   │   │   ├── JsonViewer.jsx       # JSON display (collapsible)
│   │   │   └── WireframePreview.jsx # Visual layout preview
│   │   ├── data/
│   │   │   └── initialLayout.json   # Provided design JSON
│   │   ├── hooks/
│   │   │   └── useLayoutAgent.js    # Chat + layout state logic
```

```
│   │   ├── utils/
│   │   │   └── api.js              # Axios calls to backend
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
│
├── server/                         # Backend
│   ├── routes/
│   │   └── chat.js                 # POST /api/chat
│   ├── services/
│   │   ├── llmService.js           # Anthropic SDK wrapper
│   │   └── layoutTransforms.js     # resizeArtboard, moveNode, etc.
│   ├── prompts/
│   │   └── systemPrompt.js         # Big LLM prompt
│   ├── utils/
│   │   └── jsonValidator.js        # Validate LLM output shape
│   ├── .env                        # API keys (gitignored)
│   ├── index.js                    # Express entry point
│   └── package.json
│
├── README.md                       # Setup instructions
├── APPROACH.md                     # Your short note
└── .gitignore
```

---

6. Example Code Snippets

6.1 Backend: Basic Express Setup

```
// server/index.js
import express from 'express';
import cors from 'cors';
import 'dotenv/config';
import chatRoute from './routes/chat.js';

const app = express();
app.use(cors());
app.use(express.json({ limit: '10mb' })); // JSON can be large

app.use('/api/chat', chatRoute);

app.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

6.2 The System Prompt (the heart of your agent)

```
// server/prompts/systemPrompt.js
export const buildSystemPrompt = (layout) => `
You are a layout transformation agent. You modify design layout JSON
based on natural language user instructions.

CANVAS RULES:
```

```
- The artboard defines the canvas (width × height).
- Every node has absolute (x, y, width, height) AND normalized
  (nx, ny, nw, nh) coordinates relative to the artboard.
- When you change the artboard size, recompute absolute values
  using normalized values to preserve layout proportions.

SEMANTIC ROLES (infer from name + content):
- "Background" → full-canvas image
- "Product" → main product image (usually large, center)
- "headline" → largest text, often the main message
- "offer badge" / "discount" → smaller circular elements with %
- "CTA" / "offer" → "Limited time offer"-style text

OUTPUT FORMAT (strict):
Return ONLY a JSON object with this exact shape:
{
  "explanation": "Short friendly message to the user",
  "updatedLayout": { ...full layout JSON... }
}
Do not include any text outside this JSON object.

CURRENT LAYOUT:
${JSON.stringify(layout, null, 2)}
`;
```

6.3 LLM Service Call

```
// server/services/llmService.js
import Anthropic from '@anthropic-ai/sdk';

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

export async function callLLM(systemPrompt, history, userMessage) {
  const response = await client.messages.create({
    model: 'claude-opus-4-7',
    max_tokens: 4096,
    system: systemPrompt,
    messages: [
      ...history,
      { role: 'user', content: userMessage }
    ]
  });

  // Extract text and parse JSON safely
  const text = response.content[0].text;
  return JSON.parse(text);
}
```

6.4 Layout Transformation Helper

```
// server/services/layoutTransforms.js
export function resizeArtboard(layout, newWidth, newHeight) {
  const updated = structuredClone(layout); // deep copy, never mutate input
  const rootId = updated.rootNodes[0];
  const artboard = updated.nodes[rootId];
```

```
  artboard.width = newWidth;
  artboard.height = newHeight;

  // Recompute every child from normalized coordinates
  artboard.children.forEach((childId) => {
    const node = updated.nodes[childId];
    node.x = node.nx * newWidth;
    node.y = node.ny * newHeight;
    node.width = node.nw * newWidth;
    node.height = node.nh * newHeight;
  });

  return updated;
}
```

6.5 Frontend Chat Hook

```
// client/src/hooks/useLayoutAgent.js
import { useState } from 'react';
import axios from 'axios';
import initialLayout from '../data/initialLayout.json';

export function useLayoutAgent() {
  const [layout, setLayout] = useState(initialLayout);
  const [messages, setMessages] = useState([]);
  const [loading, setLoading] = useState(false);

  const sendMessage = async (text) => {
    const newUserMsg = { role: 'user', content: text };
    setMessages((prev) => [...prev, newUserMsg]);
    setLoading(true);

    try {
      const { data } = await axios.post('<http://localhost:3001/api/chat>', {
        message: text,
        layout,
        history: messages.slice(-6) // last 6 for context
      });

      setLayout(data.updatedLayout);
      setMessages((prev) => [
        ...prev,
        { role: 'assistant', content: data.explanation }
      ]);
    } catch (err) {
      setMessages((prev) => [
        ...prev,
        { role: 'assistant', content: 'Sorry, something went wrong.' }
      ]);
    } finally {
      setLoading(false);
    }
  };

  return { layout, messages, loading, sendMessage };
```

```
}
```

6.6 Wireframe Preview Component

```
// client/src/components/WireframePreview.jsx
export default function WireframePreview({ layout }) {
  const rootId = layout.rootNodes[0];
  const artboard = layout.nodes[rootId];
  const aspectRatio = artboard.height / artboard.width;

  return (
    <div
      style={{
        position: 'relative',
        width: '100%',
        paddingBottom: `${aspectRatio * 100}%`,
        background: artboard.data?.backgroundColor || '#f0f0f0',
        border: '1px solid #ccc'
      }}
    >
      {artboard.children.map((id) => {
        const node = layout.nodes[id];
        return (
          <div
            key={id}
            style={{
              position: 'absolute',
              left: `${node.nx * 100}%`,
              top: `${node.ny * 100}%`,
              width: `${node.nw * 100}%`,
              height: `${node.nh * 100}%`,
              background: getColorForType(node.type),
              fontSize: '10px',
              padding: '2px'
            }}
          >
            {node.data?.content || node.name}
          </div>
        );
      })}
    </div>
  );
}

function getColorForType(type) {
  return {
    image: 'rgba(100, 150, 255, 0.4)',
    text: 'rgba(255, 200, 100, 0.5)',
    shape: 'rgba(255, 100, 100, 0.5)'
  }[type] || '#ddd';
}
```

6.7 JSON Validation

```
// server/utils/jsonValidator.js
```

```
export function validateLayout(layout) {
  if (!layout) throw new Error('Layout is null');
  if (!Array.isArray(layout.rootNodes)) throw new Error('rootNodes must be array');
  if (typeof layout.nodes !== 'object') throw new Error('nodes must be object');

  for (const id of layout.rootNodes) {
    if (!layout.nodes[id]) throw new Error(`Missing root node: ${id}`);
  }
  return true;
}
```

---

7. Testing & Submission Checklist

Functional Tests

- [ ] App loads with initial JSON visible
- [ ] User can send a chat message
- [ ] "Convert to 9:16" updates artboard to 1080×1920
- [ ] "Move headline to top" repositions the headline
- [ ] "Make the headline smaller" reduces font size
- [ ] Follow-up like "make it bigger" understands context
- [ ] JSON viewer updates after every change
- [ ] Wireframe preview updates after every change
- [ ] Errors are handled (LLM fails, invalid JSON returned)

README Should Include

1. Project description (2–3 sentences)
2. Prerequisites (Node version, API key)
3. Setup steps (clone, install, env vars, run)
4. How to use (example prompts to try)
5. Tech stack overview

Approach Note Should Cover

1. How you structured the LLM prompt
2. How you handle JSON transformations safely
3. How you maintain conversation context
4. Any trade-offs you made (and what you'd improve with more time)

Final Submission

1. Push to a public GitHub repo
2. Add README and APPROACH notes at root
3. (Optional) Record a 3–5 min Loom showing 4–5 example instructions
4. Share the repo link

Estimated Time Budget (24 hours)

Phase	Time

Setup + understanding JSON	1.5 h
Frontend chat UI	2 h
Backend + LLM integration	3 h
System prompt design + testing	2 h
Transformations + validation	2.5 h
Wireframe preview	1.5 h
Polish, testing, edge cases	2 h
README, approach note, Loom	1.5 h
Buffer for debugging	**4 h**

Final Tips for Beginners

1. **Start small.** Get one transformation working end-to-end (e.g., aspect ratio conversion) before adding more.
2. **Always validate LLM output.** Use try/catch around `JSON.parse` and validate the structure.
3. **Never commit your API key.** Add `.env` to `.gitignore` immediately.
4. **Iterate on the prompt.** 80% of agent quality comes from the system prompt.
5. **Combine code + LLM.** Let code do math (resizing) and let the LLM do reasoning (which element is the "headline"?).
6. **Test follow-ups early.** Conversation context is what separates a real agent from a one-shot tool.

Good luck — you've got everything you need to build this. The hardest part is the prompt and JSON validation; everything else is standard web development.